

Laboratoire 06 — Orchestrateur Saga et Distributed Tracing

LOG430-02 — Architecture logicielle, École de technologie supérieure (ÉTS)

Chargé de laboratoire	Gabriel C. Ullmann
Étudiant	Ralph Christian Gabriel
Code permanent	GABR77340401
Session	Été 2026
Application	Store Manager (suite du Labo 03)
Date des mesures	2026-06-10

Dépôt principal	log430-labo6-saga-orchestrator
Services impliqués	saga-orchestrator (5123), store-manager (5000), payments-api (5009), api-gateway KrakenD (8080), Jaeger (16686)
Patron implémenté	Saga avec orchestration centralisée + machine à états + compensation (rollback)
Observabilité	OpenTelemetry → Jaeger (tracing distribué sur les 3 microservices + la Gateway)

1. Analyse du patron Saga (Activité 1)

💡 Question 1 — Lequel de ces 3 fichiers représente la machine à états ? Son implémentation est-elle complète ?

Le fichier `src/controllers/order_saga_controller.py` représente la **machine à états**. C'est lui qui détient l'état courant (`self.current_saga_state`), exécute la boucle de transitions et décide quel Handler appeler pour chaque état. Les états eux-mêmes sont définis dans `src/order_saga_state.py` (l'énumération `OrderSagaState`), et chaque *transition* est réalisée par une classe Handler.

Dans le gabarit fourni, l'implémentation était **incomplète** : seul l'état initial (`START → CreateOrder`) était fonctionnel. Les transitions `DecreaseStock`, `CreatePayment` et `DeleteOrder` n'étaient pas branchées, et l'état `STOCK_INCREASED` ne faisait qu'un commentaire `TODO` au lieu d'appeler `DeleteOrderHandler`. Extrait du gabarit original :

```
elif self.current_saga_state == OrderSagaState.STOCK_INCREASED:
    self.logger.debug("TODO: implémentez et utilisez la classe DeleteOrderHandler ...")
    self.current_saga_state = OrderSagaState.ORDER_DELETED
```

La boucle d'états (machine à états) telle qu'elle existe après implémentation complète :

```
while self.current_saga_state is not OrderSagaState.END:
    if self.current_saga_state == OrderSagaState.START:
        self.current_saga_state = self.create_order_handler.run()
    elif self.current_saga_state == OrderSagaState.ORDER_CREATED:
        self.decrease_stock_handler = DecreaseStockHandler(self.create_order_handler.order_id, order_data['items'])
        self.current_saga_state = self.decrease_stock_handler.run()
    if self.current_saga_state == OrderSagaState.ORDER_DELETED:
        self.is_error_occurred = True
    elif self.current_saga_state == OrderSagaState.STOCK_DECREASED:
        self.create_payment_handler = CreatePaymentHandler(self.create_order_handler.order_id, order_data)
        self.current_saga_state = self.create_payment_handler.run()
    elif self.current_saga_state == OrderSagaState.STOCK_INCREASED:
        self.is_error_occurred = True
        self.delete_order_handler = DeleteOrderHandler(self.create_order_handler.order_id)
        self.current_saga_state = self.delete_order_handler.run()
    elif self.current_saga_state is OrderSagaState.PAYMENT_CREATED or self.current_saga_state is
    OrderSagaState.ORDER_DELETED:
        self.current_saga_state = OrderSagaState.END
```

Cette boucle correspond **exactement** au diagramme de machine à états (`state_machine.puml`) : chaque `elif` est un état, chaque appel de Handler est une transition.

💡 Question 2 — Le handler `CreateOrderHandler` se connecte-t-il directement à une base de données ?

Non. `CreateOrderHandler` ne touche jamais une base de données. Il effectue uniquement un appel HTTP REST vers le Store Manager via l'API Gateway (KrakenD). C'est le Store Manager qui possède la logique d'accès à MySQL/Redis. Extrait de `src/handlers/create_order_handler.py` :

```
def run(self):
    """Call StoreManager to create order"""
    response = requests.post(f'{config.API_GATEWAY_URL}/store-manager-api/orders',
                             json=self.order_data,
                             headers={'Content-Type': 'application/json'})
    if response.ok:
        data = response.json()
        self.order_id = data['order_id'] if data else 0
    return OrderSagaState.ORDER_CREATED
```

L'orchestrateur reste donc **découplé** des bases de données : il ne fait que coordonner des services. Aucune dépendance comme `mysql.connector` ou `redis` n'est importée dans l'orchestrateur (voir `requirements.txt` qui ne contient que `Flask`, `requests`, etc.).

💡 Question 3 — Quelle requête de la collection Postman du Labo 05 correspond à l'endpoint appelé par `CreateOrderHandler` ?

`CreateOrderHandler` appelle `POST {{gateway}}/store-manager-api/orders`, que KrakenD route vers le backend `POST http://store_manager:5000/orders`. Dans la collection Postman du Store Manager (`docs/collections/store_manager.json`), cela correspond à la requête « **Orders / /orders** » (méthode **POST**) :

Requête Postman (Labo 05)	Méthode	URL	Corps
Orders / /orders	POST	{{baseURL}}/orders	{ "user_id": 1, "items": [...] }

Le routage KrakenD (extrait de `krakend.json`) confirme la correspondance Gateway → backend :

```
{
  "endpoint": "/store-manager-api/orders", "method": "POST",
  "backend": [{ "url_pattern": "/orders", "host": ["http://store_manager:5000"] }]
}
```

Vérification : logs de la saga (flux nominal)

En appelant `POST localhost:5123/saga/order`, on obtient exactement la séquence d'états attendue par le document arc42 :

```
Controller - DEBUG - État initial
Handler - DEBUG - Transition d'état: CreateOrder -> ORDER_CREATED
Handler - DEBUG - Transition d'état: DecreaseStock -> STOCK_DECREASED
Handler - DEBUG - Transition d'état: CreatePayment -> PAYMENT_CREATED
Controller - DEBUG - Transition à l'état terminal
"POST /saga/order HTTP/1.1" 200
```

2. Implémentation de la gestion de stock (Activité 2)

💡 Question 4 — Quel endpoint avez-vous appelé pour modifier le stock ? Quelles informations de la commande avez-vous utilisées ?

J'ai appelé `PUT {{gateway}}/store-manager-api/stocks` (routé vers `PUT http://store_manager:5000/stocks`). J'ai utilisé la **liste des articles de la commande** (`order_item_data = items`, c.-à-d. `product_id + quantity`) avec l'opération `"-"` pour retirer du stock. Pour la compensation (rollback), j'utilise l'**ID de la commande** afin de la supprimer. Extrait de `src/handlers/decrease_stock_handler.py` :

```
def run(self):
    response = requests.put(f'{config.API_GATEWAY_URL}/store-manager-api/stocks',
        json={
            "items": self.order_item_data,
            "operation": "-"
        },
        headers={'Content-Type': 'application/json'})
    if response.ok:
        return OrderSagaState.STOCK_DECREASED
    else:
        return self.rollback()

def rollback(self):
    # Compensation : la commande a déjà été créée → on la supprime via son ID
    requests.delete(f'{config.API_GATEWAY_URL}/store-manager-api/orders/{self.order_id}',
        headers={'Content-Type': 'application/json'})
    return OrderSagaState.ORDER_DELETED
```

Vérification (stock du produit 2, avant/après une commande de quantité 5) :

Produit	Stock avant	Stock après <code>DecreaseStock</code>
2 (Keyboard DEF)	500	495

La transition `1 → 5 (DecreaseStockFailure)` du diagramme est respectée : si le retrait de stock échoue, la commande est supprimée et la saga passe à `ORDER_DELETED`.

3. Implémentation de la création de paiement (Activité 3)

💡 Question 5 — Quel endpoint avez-vous appelé pour générer une transaction de paiement ? Quelles informations avez-vous utilisées ?

Deux appels sont nécessaires. (1) Je récupère d'abord la commande via `GET {{gateway}}/store-manager-api/orders/{order_id}` pour obtenir son `total_amount`. (2) Je crée ensuite la transaction via `POST {{gateway}}/payments-api/payments` (routé vers `http://payments_api:5009/payments`) en envoyant `user_id`, `order_id` et `total_amount`. Extrait de `src/handlers/create_payment_handler.py` :

```
def run(self):
    # 1) Récupérer le total_amount de la commande
    order_response = requests.get(f'{config.API_GATEWAY_URL}/store-manager-api/orders/{self.order_id}', ...)
    order = order_response.json() or {}
    self.total_amount = order.get('total_amount', 0)

    # 2) Créer la transaction de paiement
    payment_response = requests.post(f'{config.API_GATEWAY_URL}/payments-api/payments',
        json={
            "user_id": self.order_data.get('user_id'),
            "order_id": self.order_id,
            "total_amount": self.total_amount
```

```

    }, ...)
    if payment_response.ok:
        return OrderSagaState.PAYMENT_CREATED
    else:
        return self.rollback()

def rollback(self):
    # Compensation : on remet en stock (opération "+") les articles retirés
    requests.put(f'{config.API_GATEWAY_URL}/store-manager-api/stocks',
        json={"items": self.order_data.get('items', []), "operation": "+"}, ...)
    return OrderSagaState.STOCK_INCREASED

```

Vérification : pour une commande (produit 3 × 2, produit 2 × 4) le Store Manager calcule `total_amount = 249.50`, et la transaction de paiement créée dans `payments-api` porte bien `order_id=1, total_amount=249.50`. La transition `2 → 4` (`CreatePaymentFailure`) est respectée : en cas d'échec, le stock est restauré et la saga passe à `STOCK_INCREASED`.

4. Implémentation de la suppression de la commande (Activité 4)

`DeleteOrderHandler` est le seul handler déclenché **uniquement en cas d'erreur** ; il n'effectue que la suppression de la commande et n'a pas de `rollback` (conforme au diagramme). Il est appelé depuis le contrôleur à l'état `STOCK_INCREASED` (transition `4 → 5 : DeleteOrder`). Extrait de `src/handlers/delete_order_handler.py` :

```

def run(self):
    # Supprime la commande via son ID (transition 4 -> 5 du diagramme)
    requests.delete(f'{config.API_GATEWAY_URL}/store-manager-api/orders/{self.order_id}',
        headers={'Content-Type': 'application/json'})
    return OrderSagaState.ORDER_DELETED

```

Et son branchement dans `order_saga_controller.py` (l'appel manquant du gabarit) :

```

elif self.current_saga_state == OrderSagaState.STOCK_INCREASED:
    self.is_error_occurred = True
    self.delete_order_handler = DeleteOrderHandler(self.create_order_handler.order_id)
    self.current_saga_state = self.delete_order_handler.run()

```

Note (compensation des compensations) : conformément à l'énoncé, il n'y a pas de « rollback sur les rollbacks ». Chaque `rollback()` est encapsulé dans un `try/except` qui journalise l'erreur et poursuit vers l'état suivant jusqu'à l'état terminal.

5. Intégration de Jaeger — Distributed Tracing (Activité 5)

J'ai ajouté le service `jaeger` au `docker-compose.yml` de l'orchestrateur, ajouté les 5 dépendances OpenTelemetry aux `requirements.txt` des 3 services, et instrumenté chacun d'eux (orchestrateur, store-manager, payments-api). KrakenD est configuré pour exporter ses traces vers Jaeger et propager le contexte (`input_headers: ["*"]` + bloc `telemetry/opentelemetry`). Configuration OTEL commune (exemple, orchestrateur) :

```
resource = Resource.create({"service.name": "saga-orchestrator", "service.version": "1.0.0"})
trace.set_tracer_provider(TracerProvider(resource=resource))
tracer = trace.get_tracer(__name__)
otlp_exporter = OTLPSpanExporter(endpoint="http://jaeger:4317", insecure=True)
trace.get_tracer_provider().add_span_processor(BatchSpanProcessor(otlp_exporter))
FlaskInstrumentor().instrument_app(app) # instrumentation auto de Flask
RequestsInstrumentor().instrument()      # instrumentation auto des appels sortants

@app.post('/saga/order')
def saga_order():
    with tracer.start_as_current_span("saga-order"): # span explicite
        ...
```

Point technique résolu : KrakenD 2.7 n'acceptait pas le contexte de trace entrant car l'OpenTelemetry SDK récent émet le *traceparent* avec le flag W3C niveau 2 (« random », valeur `03`), que la Gateway ignore. La trace de l'orchestrateur restait donc isolée. La correction consiste à fixer les paquets OTEL à une version émettant le flag `01` et à activer `OTEL_PROPAGATORS=tracecontext,baggage` sur KrakenD. Après cela, **une seule trace** relie les 4 services.

Trace complète d'une saga réussie (Jaeger)

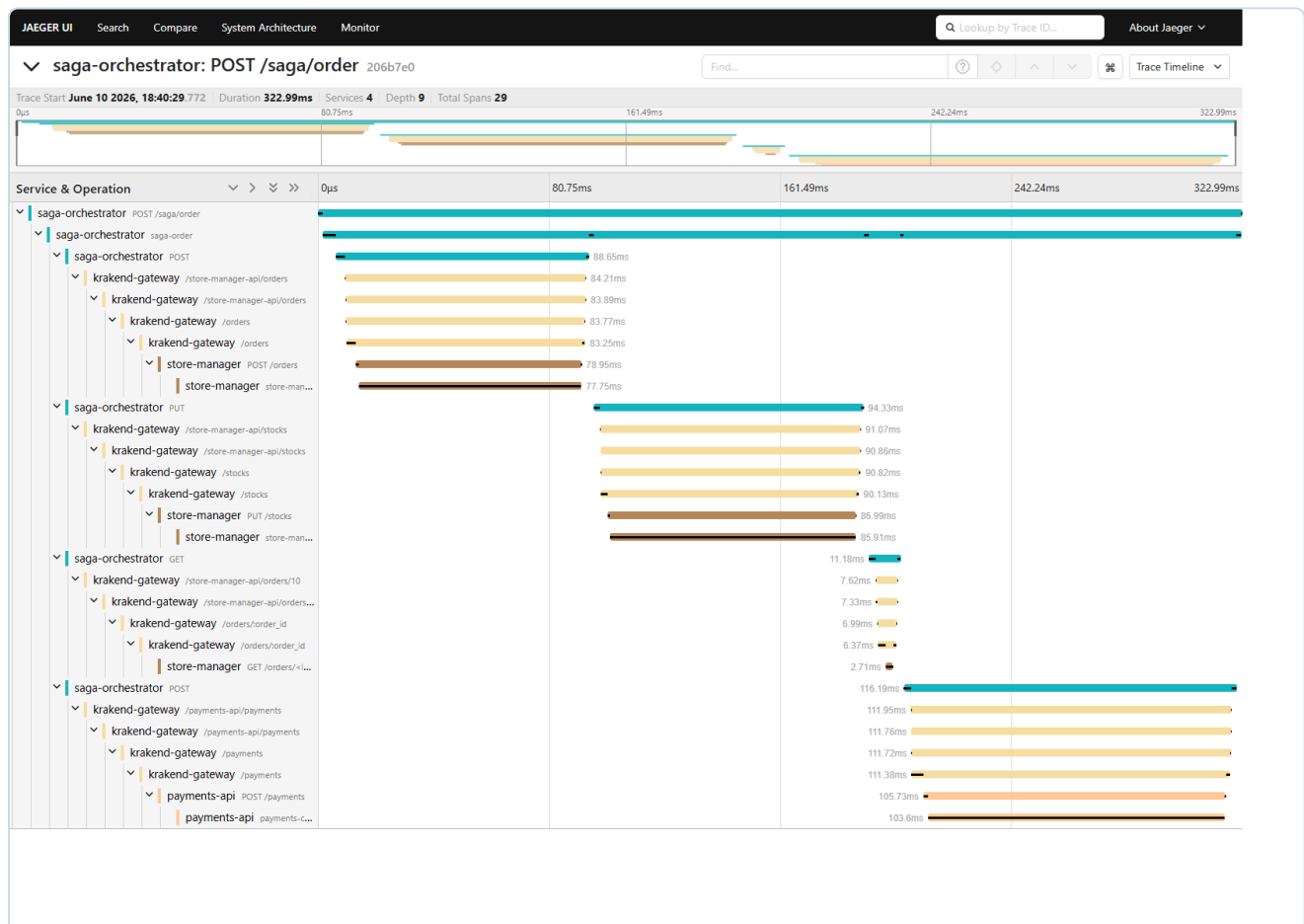


Figure 1 — Trace distribuée d'une saga nominale : **4 services, 9 niveaux de profondeur, 29 spans**. On voit le flux complet `saga-orchestrator` → `krakend-gateway` → `store-manager` (POST /orders, PUT /stocks, GET /orders) puis → `payments-api` (POST /payments), le tout dans une seule trace liée.

Vue de recherche Jaeger (les 3 services + Gateway)

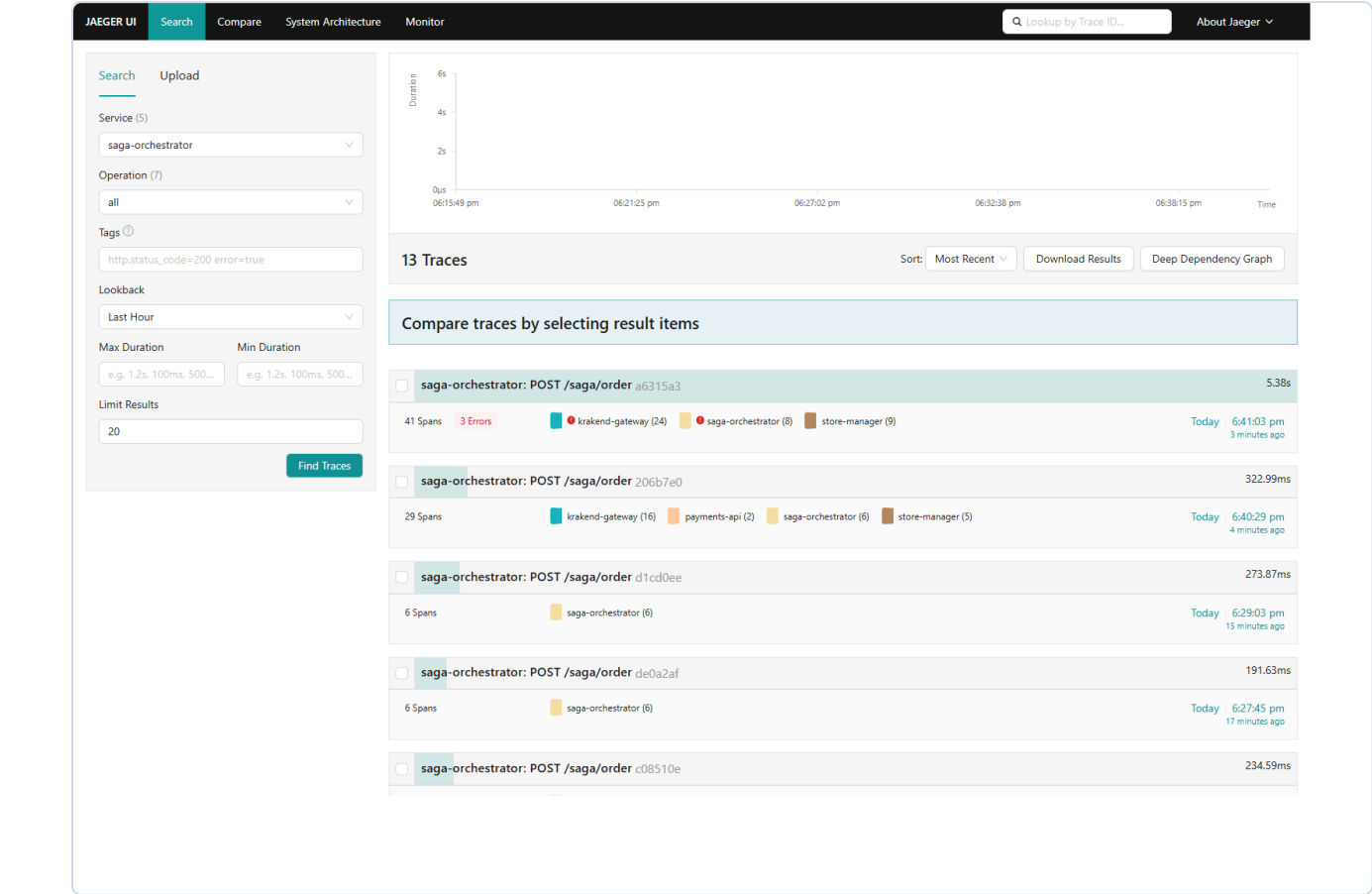


Figure 2 — La liste des traces montre, pour chaque saga, la répartition des spans par service (`krakend-gateway` , `payments-api` , `saga-orchestrator` , `store-manager`). La trace en échec apparaît avec « 3 Errors ».

6. Test de la transaction Saga complète (Activité 6)

Question 6 — Quelle est la différence entre appeler l'orchestrateur Saga et appeler directement les endpoints des services individuels ? Avantages / inconvénients ?

Appeler directement les endpoints (POST /orders, PUT /stocks, POST /payments) oblige le **client** à connaître la séquence, à gérer les erreurs et à compenser manuellement si une étape échoue ; un échec partiel laisse le système **incohérent** (commande sans stock décrémenté ni paiement). Appeler l'orchestrateur (`POST /saga/order`) centralise toute cette logique : le client fait **un seul appel**, et l'orchestrateur garantit l'atomicité logique (tout réussit, ou tout est compensé).

Approche	Avantages	Inconvénients
Orchestrateur Saga <code>POST /saga/order</code>	- Cohérence garantie (compensation automatique) - Logique centralisée et réutilisable - Client simple (1 appel) - Observabilité d'un flux de bout en bout	- Point de défaillance unique (l'orchestrateur) - Couplage temporel (appels synchrones) - Latence cumulée des étapes
Appels directs aux 3 services	- Moins de composants - Latence d'un appel isolé plus faible	- Risque d'état incohérent en cas d'échec partiel - Logique de coordination dupliquée chez chaque client - Compensation manuelle et fragile

Concrètement, la même commande passée par l'orchestrateur produit, en un appel, la création de la commande, la décrémentement du stock et la transaction de paiement — vérifiées dans chaque service :

Service	Vérification	Résultat
store-manager	Commande créée (MySQL + Redis)	<code>order_id=10, total_amount=59.50</code>
store-manager	Stock décrémenté	<code>produit 2 : 500 → 495 → ... cohérent</code>
payments-api	Transaction de paiement	<code>order_id correspondant, total_amount correct</code>
Jaeger	Trace de bout en bout	<code>4 services, 29 spans (Figure 1)</code>

7. Gestion des échecs et compensation (Activité 7)

En **arrêtant le service payments-api** puis en appelant la saga, l'étape `CreatePayment` échoue (HTTP 500 renvoyé par la Gateway). La saga déclenche alors la compensation : restauration du stock (`STOCK_INCREASED`) puis suppression de la commande (`DeleteOrder` → `ORDER_DELETED`), exactement comme dans le diagramme. Logs observés :

```

Controller - DEBUG - État initial
Handler    - DEBUG - Transition d'état: CreateOrder      -> ORDER_CREATED
Handler    - DEBUG - Transition d'état: DecreaseStock      -> STOCK_DECREASED
Handler    - ERROR  - CreatePayment a échoué : 500
Handler    - DEBUG - Transition d'état: CreatePaymentFailure -> STOCK_INCREASED
Handler    - DEBUG - Transition d'état: DeleteOrder        -> ORDER_DELETED
Controller - DEBUG - Transition à l'état terminal
"POST /saga/order HTTP/1.1" 500

```

Effets vérifiés (la commande est annulée, le stock revient à son état initial) :

Élément	Avant compensation	Après compensation
Commande (order_id=11)	créée	<code>supprimée (MySQL + API renvoie {})</code>
Stock produit 2	495 (décrémenté)	<code>495 (restauré, net nul)</code>
Réponse HTTP	—	<code>500 + message d'erreur</code>

Trace de la compensation dans Jaeger

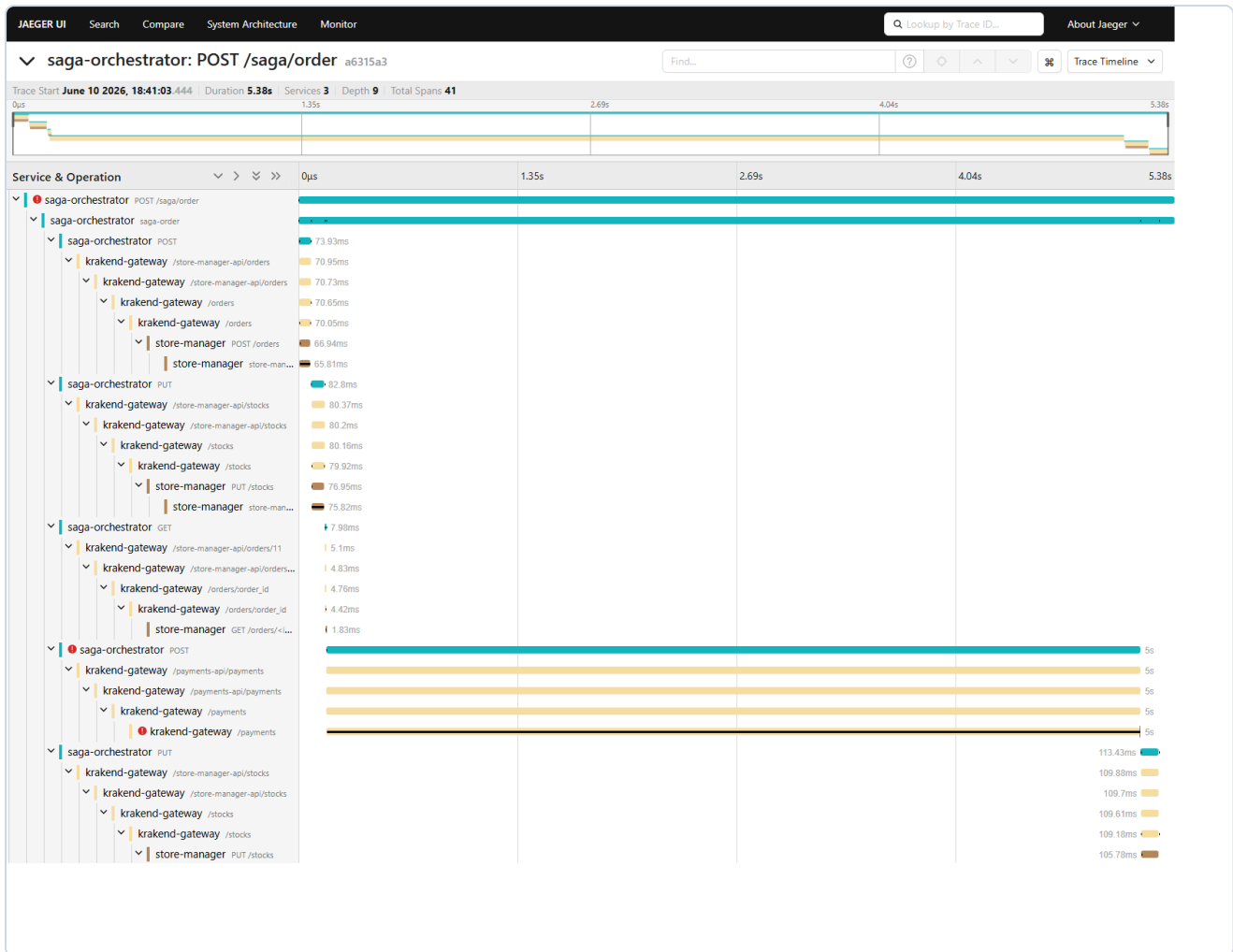


Figure 3 — Trace d'une saga en échec (41 spans, 3 erreurs). Les marqueurs rouges  signalent l'échec de `POST /payments` (timeout 5 s, service arrêté). On voit ensuite la transition de compensation `PUT /store-manager-api/stocks` (remise en stock) puis la suppression de la commande, le tout tracé de bout en bout.

8. Conclusion

Le patron **Saga orchestré** est entièrement implémenté : les 4 transitions (CreateOrder, DecreaseStock, CreatePayment, DeleteOrder) et leurs **compensations** (suppression de commande, restauration de stock) suivent fidèlement la machine à états du document arc42, vérifiées sur le flux nominal et le flux d'échec. Le **tracing distribué** via OpenTelemetry/Jaeger relie l'orchestrateur, le store-manager et le payments-api (+ la Gateway) dans une seule trace, permettant de visualiser et déboguer chaque transaction de bout en bout. L'orchestrateur reste découplé des bases de données : il ne fait que coordonner des appels HTTP REST, garantissant la cohérence des données sans transactions ACID distribuées.